

목차

1장 기본값이 하는 일	2
1.1 기본값의 정의	2
1.2 기본값이 정하는 것	3
1.3 안전한 기본값	4
1.4 편한 기본값과 안전한 기본값	5
1.5 기본값이 감추는 것	6
1.6 값이 정해지는 자리	7
1.7 기본값을 적어 두는 자리	8
2장 설정과 우선순위	9
2.1 설정의 총	9
2.2 총을 정하는 순서	10
2.3 어디서 온 값인지 알기	11
2.4 바꿀 수 있는 값과 없는 값	12
2.5 설정이 겹치는 모양	13
2.6 설정이 늘어나는 문제	14
2.7 설정을 바꾸는 절차	15
3장 실패를 다루는 경계	16
3.1 실패 경계의 정의	16
3.2 빠른 실패	17
3.3 기다림의 상한	18
3.4 다시 해 보기	19
3.5 실패가 번지는 길	20
3.6 되돌리기와 남는 상태	21
3.7 실패를 감추는 기본값	22
4장 줄여서 버티기	23
4.1 다 죽는 대신 덜 주기	23
4.2 무엇을 먼저 끝 것인가	24
4.3 부하가 넘칠 때	25
4.4 부하에 따른 응답의 단계	26
4.5 줄여서 주는 응답	27
4.6 저하 상태를 알리는 방법	28
4.7 저하가 굳어질 때	29
5장 경계를 정하는 값들	30
5.1 값을 정하는 근거	30
5.2 처음 값과 조정	31
5.3 값을 실제로 확인하기	32
5.4 값이 서로 얹힐 때	33
5.5 환경마다 다른 값	34
5.6 값이 낡는 방식	35
5.7 값을 기록하는 자리	36
6장 한 연동의 경계를 정한 기록	37
6.1 연동의 상황	37
6.2 무엇부터 정했나	38
6.3 없어도 되는 부분 정하기	39
6.4 실패를 어떻게 알렸나	40
6.5 값이 실제로 걸리는지 확인	41
6.6 다시 겪은 날	42
6.7 남긴 값과 근거	43
7장 경계 설계 판단	44
7.1 얼마나 방어할 것인가	44
7.2 조용한 실패를 없애기	45
7.3 기본값을 바꾸는 순서	46
7.4 경계 설계 점검 항목	47
7.5 값을 다시 볼 때	48
7.6 저하와 중단 사이	49
7.7 핵심 원칙 정리	50

1.1 기본값의 정의

기본값은 쓰는 쪽이 아무것도 정하지 않았을 때 대신 쓰이는 값이다. 기다릴 최대 시간, 한 번에 가져올 건수, 다시 시도할 횟수처럼 정하지 않아도 프로그램이 돌아가게 해 주는 값들이 여기 든다.

기본값이 있어야 하는 이유는 단순하다. 모든 값을 쓰는 쪽이 정하게 하면 쓰기가 어려워지고, 대부분의 경우 같은 값을 반복해 적게 된다. 그래서 흔한 경우에 맞는 값을 미리 정해 두고 다른 경우에만 바꾸게 한다.

문제는 기본값이 아무 값이나 되어서는 안 된다는 데 있다. 정하지 않은 사람은 그 값이 무엇인지 모르는 채로 그 값의 결과를 받는다. 이 책은 그런 값을 어떻게 고르고, 그 값이 감당하지 못하는 상황에서 무엇이 일어나게 할지를 다룬다.

1.2 기본값이 정하는 것

기본값 하나는 값 하나가 아니라 동작 하나를 정한다. 기다릴 시간을 정해 두지 않으면 응답이 오지 않을 때 언제까지 기다릴지가 정해지지 않고, 그러면 부르는 쪽이 붙잡혀 있는 시간도 정해지지 않는다.

그래서 기본값을 고를 때는 그 값이 정하는 동작을 먼저 적는다. 이 값이 크면 무엇이 일어나고 작으면 무엇이 일어나는지를 적어 보면 어느 쪽 위험이 큰지 드러난다. 대개 한쪽은 느려지고 다른 쪽은 실패한다.

값이 정하는 것이 여러 개일 때도 있다. 한 번에 가져올 건수는 응답 시간과 사용하는 메모리와 저장소의 부담을 함께 정한다. 이런 값은 어느 하나만 보고 정하면 다른 자리에서 문제가 나므로, 정하기 전에 걸리는 자리를 모두 적는다.

1.3 안전한 기본값

기본값은 그 값을 정하지 않은 사람, 곧 이 값에 대해 가장 적게 아는 사람에게 적용된다. 그래서 기본값은 최선의 성능이 아니라 가장 덜 위험한 쪽으로 고른다.

덜 위험한 쪽을 고르는 방법은 잘못됐을 때의 결과를 견주는 것이다. 기다릴 시간을 짧게 잡으면 가끔 실패하고, 길게 잡으면 붙잡힌 자원이 쌓여 전체가 멈출 수 있다. 앞쪽은 국소적이고 뒤쪽은 전체적이므로 짧은 쪽이 기본값에 맞다.

권한이나 노출과 관련된 값은 더 분명하다. 정하지 않았을 때 넓게 허용하는 기본값은 잊고 넘어간 자리마다 구멍을 만든다. 좁게 두면 필요한 사람이 넓히면서 그 사실이 기록에 남는다. 모르는 상황에서는 좁고 짧은 쪽이 기본이다.

1.4 편한 기본값과 안전한 기본값

안전한 값이 늘 편한 값은 아니다. 좁게 잡은 권한, 짧게 잡은 대기 시간, 작게 잡은 건수는 처음 쓰는 사람에게 불편하다. 그래서 쓰기 쉽게 만들려는 힘이 기본값을 늘 넓고 길게 밀어낸다.

이 힘을 다루는 방법은 편의를 다른 자리에서 주는 것이다. 기본값은 안전하게 두고, 흔한 경우에 맞는 설정 묶음을 따로 제공해 한 줄로 고를 수 있게 하는 방식이다. 이렇게 하면 편의를 얻는 사람이 자기가 무엇을 골랐는지 알게 된다.

구분의 기준은 잊었을 때 무엇이 일어나는가다. 잊어서 손해가 생기는 값은 안전한 쪽으로 두고, 잊어도 불편할 뿐인 값은 편한 쪽으로 둔다. 모든 기본값을 안전하게 잡으려 하면 쓰기 어려워지고, 그러면 사람들이 설정을 통째로 복사해 쓰기 시작한다.

1.5 기본값이 감추는 것

기본값의 대가는 그 값이 코드에 보이지 않는다는 점이다. 설정을 읽어도 정하지 않은 값은 나타나지 않으므로, 실제 동작을 알려면 기본값이 무엇인지 따로 알아야 한다.

그래서 문제가 생겼을 때 조사하는 사람은 적혀 있는 값만 보고 판단한다. 적히지 않은 값이 원인일 때는 그 자리를 아예 의심하지 않는다. 기본값이 바뀌었거나 다른 환경에서 다른 기본값이 쓰였을 때 특히 찾기 어렵다.

이 문제를 줄이는 방법은 실제로 쓰인 값을 시작할 때 한 번 남기는 것이다. 정한 값이든 기본값이든 지금 무엇이 쓰이고 있는지 적어 두면, 조사하는 사람이 적히지 않은 값을 짐작하지 않아도 된다. 적히지 않은 값을 볼 수 있게 만드는 일이 기본값 설계의 절반이다.

값이 정해지는 자리

아무도 정하지 않으면 맨 아래가 쓰인다

먼저

실행할 때 준 값

환경에 둔 값

설정 파일의 값

☐

실제로
쓰이는 값

나중

기본값

☐

가려진 값

어느 층에서 왔는지 말할 수 있어야 한다

1.7 기본값을 적어 두는 자리

기본값은 값만 적어서는 부족하다. 왜 그 값인지 함께 적어야 나중에 바꿀지 판단할 수 있다. 근거가 없으면 그 값은 건드리면 안 되는 값처럼 굳는다.

적을 것은 세 가지다. 값 자체, 그 값을 고른 근거, 그리고 이 값이 감당하지 못하는 상황이다. 세 번째가 특히 쓸모 있다. 어떤 경우에 이 값을 바꿔야 하는지 적혀 있으면 쓰는 쪽이 자기 상황이 그 경우인지 판단할 수 있다.

적는 자리는 값이 정의된 곳 가까이가 좋다. 멀리 떨어진 문서에 적으면 값이 바뀔 때 함께 갱신되지 않는다. 값과 근거가 나란히 있으면 값을 고치는 사람이 근거도 함께 본다. 근거가 함께 있는 값만 상황이 달라졌을 때 다시 계산할 수 있다.

2.1 설정의 층

실제 시스템에서 값은 한 자리에서만 정해지지 않는다. 코드 안의 기본값이 있고, 설정 파일이 있고, 환경에 둔 값이 있고, 실행할 때 넘긴 값이 있다. 같은 값이 여러 자리에 있으면 어느 것이 쓰일지 정하는 규칙이 필요하다.

이 규칙을 우선순위라 한다. 보통은 더 구체적인 자리, 곧 더 나중에 더 가까이에서 정한 값이 이긴다. 실행할 때 넘긴 값이 가장 세고 코드 안의 기본값이 가장 약하다.

층을 두는 이유는 바꾸는 비용이 다르기 때문이다. 코드 안의 값을 바꾸려면 다시 배포해야 하지만 환경에 둔 값은 다시 시작하는 것으로 끝난다. 자주 바뀌는 값일수록 바꾸기 쉬운 층에 두는 것이 기본 배치다. 층을 몇 개까지 둘지도 함께 정한다. 층이 많을수록 값의 출처를 찾기 어려워진다.

2.2 층을 정하는 순서

우선순위는 정해 두기만 하면 되는 것이 아니라 모든 값에 맞게 적용되어야 한다. 어떤 값은 환경이 이기고 어떤 값은 파일이 이기면, 쓰는 쪽은 값마다 규칙을 외워야 한다.

규칙을 하나로 유지하는 방법은 값을 읽는 자리를 한군데로 모으는 것이다. 각 코드가 필요할 때마다 직접 환경을 들여다보면 그 자리마다 다른 순서가 생긴다. 한 자리에서 층을 합쳐 최종 값을 만들고, 나머지 코드는 그 결과만 쓴다.

합칠 때는 값 단위로 합친다. 설정 묶음 전체를 통째로 덮으면 위층에 적힌 값 하나 때문에 아래층의 나머지 값이 모두 사라진다. 이 실수는 값이 조용히 기본값으로 돌아가게 만들어 찾기 어렵다. 규칙이 하나면 쓰는 쪽은 값 하나만 확인해도 어느 층이 이겼는지 안다.

2.3 어디서 온 값인지 알기

설정 문제의 대부분은 값이 틀린 것이 아니라 예상과 다른 층의 값이 쓰인 것이다. 파일에 적었는데 환경의 값이 이기고 있거나, 환경에 적었는데 이름이 달라 기본값이 쓰이는 식이다.

그래서 지금 쓰이는 값과 그 값이 어느 층에서 왔는지를 함께 볼 수 있어야 한다. 시작할 때 최종 설정을 출처와 함께 남기면 이 확인이 한 번에 끝난다. 값을 남길 때 비밀번호나 열쇠는 가린다.

출처를 볼 수 없으면 조사하는 사람은 층을 하나씩 뒤횈다. 층이 넷이면 네 자리를 확인해야 하고, 그중 하나에 오타가 있으면 그 자리를 눈으로 찾아야 한다. 출처를 남기는 일은 이 시간을 통째로 없앤다. 출처가 함께 보이면 설정 문제의 대부분은 한 번 보는 것으로 끝난다.

2.4 바꿀 수 있는 값과 없는 값

모든 값을 밖에서 바꿀 수 있게 하면 설정이 수백 개가 된다. 반대로 아무것도 못 바꾸게 하면 상황이 달라질 때마다 다시 배포해야 한다. 그래서 어느 값을 밖으로 내보낼지 고른다.

밖으로 내보낼 값은 두 종류다. 환경마다 달라야 하는 값과, 문제가 생겼을 때 급히 조절해야 하는 값이다. 접속 주소나 자원의 크기가 앞쪽이고, 대기 시간이나 동시 처리 수가 뒤쪽이다.

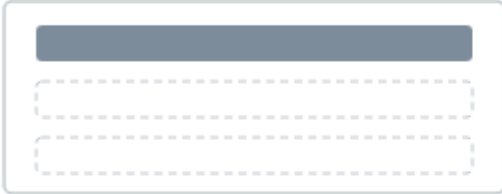
나머지는 안에 둔다. 업무 규칙에서 오는 값을 설정으로 빼면 그 규칙이 어디에 있는지 알 수 없게 되고, 값을 바꿔 규칙을 바꾸는 일이 기록 없이 일어난다. 설정으로 뺄지 판단할 때는 이 값이 규칙인지 환경인지를 먼저 묻는다. 밖으로 낸 값에는 누가 언제 바꿀 수 있는지도 함께 정해 둔다.

설정이 겹치는 모양

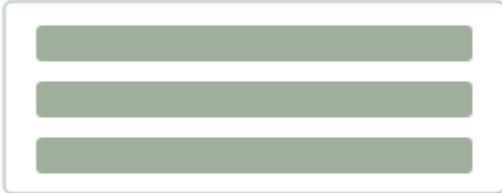
값 단위로 겹치는가 묶음 단위로 덮는가

값 단위로 겹치기

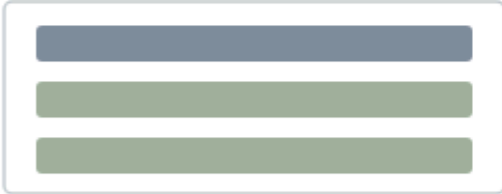
환경에 둔 값



설정 파일의 값

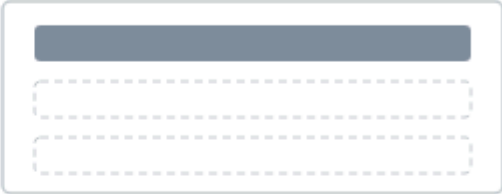


합친 결과

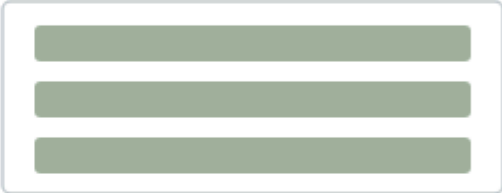


묶음 단위로 덮기

환경에 둔 값



설정 파일의 값



합친 결과



묶음을 통째로 덮었다

조용히 기본값으로 돌아가는 자리가 여기도

2.6 설정이 늘어나는 문제

설정은 늘리기 쉽고 줄이기 어렵다. 새 상황이 생길 때마다 값을 하나 빼면 그 값은 계속 남는다. 몇 년이 지나면 아무도 쓰지 않는 설정이 절반을 넘는다.

설정이 많아지면 세 가지가 나빠진다. 조합의 수가 늘어 시험할 수 없게 되고, 어떤 조합이 실제로 쓰이는지 알 수 없어지며, 값 사이의 관계가 어긋나도 아무도 모른다. 대기 시간이 재시도 간격보다 짧게 설정되는 식의 어긋남이 여기서 생긴다.

정리는 실제로 어떤 값이 기본값과 다르게 설정되어 있는지 세는 데서 시작한다. 어느 환경에서도 기본값 그대로인 설정은 지워도 된다. 지우기 전에 그 값을 기본값으로 굳히고 한동안 지켜보는 절차를 둔다. 쓰이지 않는 설정은 지우기 전까지 계속 시험과 문서의 몫을 차지한다.

2.7 설정을 바꾸는 절차

설정 변경은 코드 변경보다 가볍게 여겨지지만 결과는 같을 수 있다. 그래서 절차를 둔다. 바꾸기 전에 지금 값이 무엇이고 어느 층에서 오는지 확인하고, 바꾼 뒤에 실제로 그 값이 쓰이는지 확인한다.

확인이 필요한 이유는 층이 여럿이기 때문이다. 파일을 고쳤는데 환경의 값이 이기고 있으면 아무 일도 일어나지 않는다. 바꿨는데 동작이 그대로면 값이 잘못된 것이 아니라 층이 잘못된 것일 수 있다.

되돌리는 방법도 함께 정해 둔다. 특히 급히 조절하는 값은 바꾼 뒤 문제가 생겼을 때 빠르게 되돌릴 수 있어야 한다. 되돌릴 수 없는 변경은 급할 때 쓰는 값에서 빼는 편이 낫다. 가벼워 보이는 변경일수록 확인 절차를 적어 두는 편이 낫다.

3.1 실패 경계의 정의

실패 경계는 어떤 실패가 그 선을 넘어 번지지 못하게 하는 자리다. 다른 시스템이 응답하지 않을 때 그 영향이 이쪽 전체로 퍼지지 않게 하려면, 어디까지가 그 실패의 영향 범위인지 정해 두어야 한다.

경계를 정한다는 것은 세 가지를 정한다는 뜻이다. 무엇이 실패로 판정되는가, 실패했을 때 무엇을 돌려주는가, 그리고 실패가 이어질 때 어떻게 하는가다. 셋 가운데 하나라도 정하지 않으면 그 자리는 상황에 따라 다르게 동작한다.

경계가 없으면 실패는 기다림으로 나타난다. 응답하지 않는 쪽을 계속 기다리는 사이 이쪽의 자원이 묶이고, 묶인 자원이 다 떨어지면 관계없는 요청까지 처리되지 않는다. 한 자리의 실패가 전체 정지가 되는 혼한 경로가 이것이다.

3.2 빠른 실패

빠른 실패는 성공할 가능성이 낮은 일을 오래 붙들지 않고 일찍 실패로 끝내는 방식이다. 조건이 갖춰지지 않았으면 시작하지 않고, 기다려도 소용없는 상황이면 기다림을 끊는다.

이 방식의 이득은 자원을 돌려준다는 데 있다. 어차피 실패할 요청이 자원을 오래 쥐고 있으면 성공할 수 있는 요청까지 밀린다. 일찍 끝내면 그 자리를 다른 요청이 쓴다. 실패의 수는 늘지만 전체가 멈추지는 않는다.

빠른 실패가 맞지 않는 경우도 있다. 조금만 기다리면 성공할 일을 곧바로 실패로 끝내면 헛된 실패가 늘어난다. 그래서 얼마나 기다릴지를 정하는 일이 함께 필요하다. 다음 절이 그 상한을 다룬다. 실패의 수가 느는 것과 전체가 멈추는 것은 견줄 대상이 아니다.

3.3 기다림의 상한

기다림의 상한은 응답이 오지 않을 때 언제까지 기다릴지 정한 값이다. 이 값이 없으면 부르는 쪽은 무한정 기다리고, 기다리는 동안 잡고 있는 자원도 돌아오지 않는다.

값을 정할 때는 정상일 때의 응답 시간 분포를 본다. 대부분이 백 밀리초 안에 끝나는데 상한을 삼십 초로 잡으면 사실상 상한이 없는 것과 같다. 상위 구간보다 조금 넉넉한 값이 실용적인 출발점이다.

상한은 총마다 함께 본다. 바깥 요청의 상한이 오 초인데 안에서 부르는 곳의 상한이 십 초면 안쪽 상한은 절대 걸리지 않는다. 안쪽이 바깥보다 짧아야 안쪽에서 먼저 끊기고, 그래야 어디서 끊겼는지 알 수 있다. 상한이 없다는 말은 상한을 도구가 대신 정한다는 말과 같다.

3.4 다시 해 보기

일시적인 실패는 다시 해 보면 성공한다. 그래서 재시도는 쓸모가 있지만 규칙 없이 쓰면 상황을 악화시킨다. 상대가 힘들어서 실패했는데 곧바로 다시 부르면 더 힘들어진다.

그래서 세 가지를 정한다. 몇 번까지 다시 할지, 사이에 얼마를 쉬지, 그리고 어떤 실패에만 다시 할지다. 쉬는 시간은 갈수록 늘리는 편이 낫고, 여러 곳이 동시에 다시 부르지 않도록 조금씩 다르게 두는 방법도 쓴다.

어떤 실패에 다시 할지가 특히 중요하다. 잘못된 요청을 다시 보내면 몇 번을 해도 실패한다. 이쪽의 잘못으로 실패한 것은 다시 하지 않고, 상대가 잠시 어려워 실패한 것만 다시 한다. 두 실패를 구분해 알려 주는 것이 앞선 권들에서 다룬 실패 표현의 값이다.

실패가 번지는 길

기다림이 길면 실패가 위로 번진다

자원이 모자라 다른 요청도 멈춘다



막는 자리를 정하면 위층은 계속 답한다

3.6 되돌리기와 남는 상태

여러 단계로 이루어진 처리가 중간에서 실패하면 앞 단계의 결과가 남는다. 값을 저장했고 바깥에 알렸는데 마지막 단계에서 실패한 경우다. 이때 무엇을 되돌리고 무엇을 남길지 정해 두지 않으면 상태가 매번 다르게 남는다.

되돌릴 수 있는 것은 되돌린다. 같은 저장소 안의 변경은 대개 함께 되돌릴 수 있다. 되돌릴 수 없는 것은 바깥에 이미 나간 요청과 알림이다. 그래서 되돌릴 수 없는 일은 되도록 마지막에 둔다.

되돌릴 수 없는 자리가 앞에 있어야 한다면, 같은 일이 두 번 일어나도 결과가 같게 만든다. 그러면 다시 시도해도 안전해진다. 부분 실패는 없앨 수 없으므로 남는 상태를 예상할 수 있게 만드는 것이 설계의 목표다.

3.7 실패를 감추는 기본값

실패했을 때 빈 목록이나 영을 돌려주면 화면은 정상으로 보인다. 오류도 나지 않고 사용자도 아무 말을 듣지 않는다. 그래서 이 방식은 편해 보이지만 실패를 없애는 것이 아니라 감춘다.

감춰진 실패는 두 가지 비용을 만든다. 첫째, 잘못된 값이 그대로 다음 처리에 쓰인다. 재고가 조회되지 않아 영으로 나온 것과 실제로 영인 것은 다른데 구분할 수 없다. 둘째, 문제가 있다는 사실을 아무도 모른다.

그래서 실패를 대신할 값을 쓸 때는 그 값이 실패에서 왔다는 사실을 함께 전한다. 화면에 지금 일부를 보여 주지 못한다고 알리거나, 응답에 그 자리가 비어 있는 이유를 담는다. 대신하는 값과 실제 값을 구분할 수 없게 만드는 것이 가장 나쁜 선택이다.

4.1 다 죽는 대신 덜 주기

어떤 부분이 동작하지 않을 때 선택은 둘이다. 전체를 실패로 끝내거나, 되는 부분만 주고 나머지를 포기하는 것이다. 뒤쪽을 점진적 저하라 한다.

이 선택이 가능한 이유는 화면이나 응답의 부분들이 중요도가 다르기 때문이다. 상품 목록은 없으면 아무 일도 못 하지만 추천 목록은 없어도 나머지가 쓸모 있다. 없어도 되는 부분을 미리 정해 두면 그 부분이 실패해도 전체가 살아난다.

저하는 공짜가 아니다. 어느 부분이 빠졌는지 알려야 하고, 빠진 채로 동작하는 코드가 필요하며, 빠진 상태가 오래가지 않게 지켜봐야 한다. 그래서 모든 부분을 저하 대상으로 만들지 않고 몇 개만 고른다. 고른 부분이 적을수록 저하가 실제로 동작할 가능성이 높다.

4.2 무엇을 먼저 끝 것인가

저하를 쓰려면 무엇을 먼저 끝지 미리 정해야 한다. 문제가 생긴 뒤에 정하면 급한 판단이 되고, 급한 판단은 중요한 것을 끄는 실수를 만든다.

순서를 정하는 기준은 그 부분 없이도 사용자가 하려던 일을 끝낼 수 있는가다. 끝낼 수 있으면 끝 수 있고, 없으면 끝 수 없다. 이 기준으로 보면 대개 보조 정보, 추천, 통계, 개인화가 먼저 꺼진다.

정한 순서는 적어 두고 실제로 꺼 본다. 적어만 두고 시험하지 않으면 정작 필요한 때에 꺼지지 않는 부분이 나온다. 끝 수 있다고 적힌 부분이 실제로는 다른 처리에 필요한 값을 만들고 있는 경우가 흔하다. 끝 순서를 정하는 일은 무엇이 핵심인지 다시 정하는 일이기도 하다.

4.3 부하가 넘칠 때

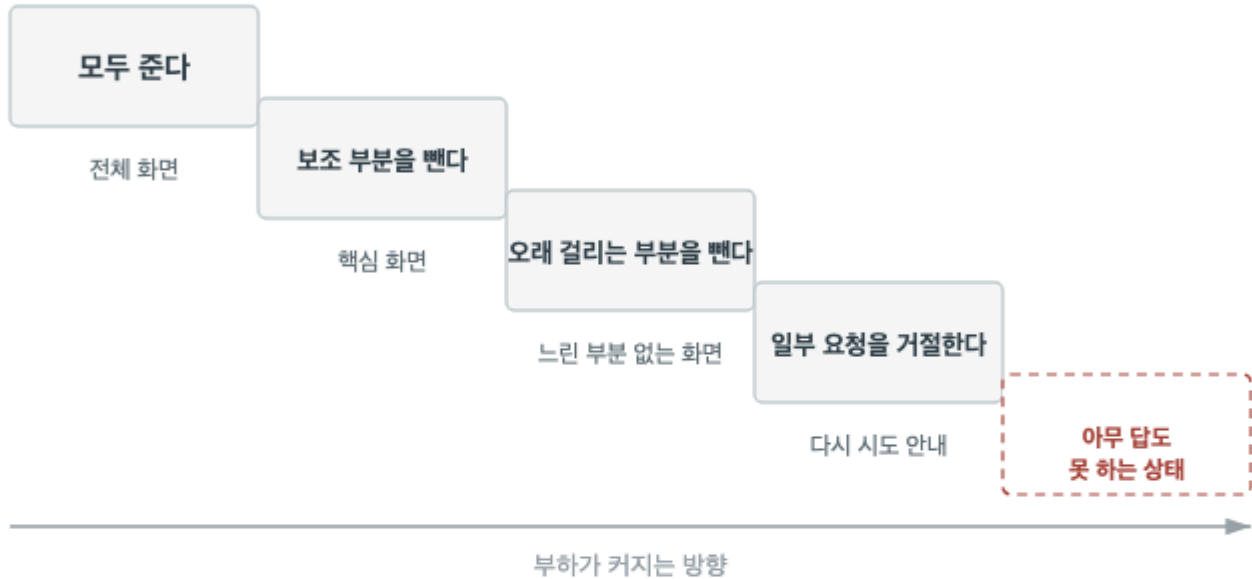
요청이 감당할 수 있는 양을 넘으면 모두를 처리하려는 시도가 오히려 아무것도 처리하지 못하게 만든다. 대기줄이 길어지면 기다리는 동안 요청이 이미 의미를 잃고, 그렇게 버려질 요청 때문에 새 요청도 밀린다.

그래서 넘칠 때는 받는 양을 줄인다. 대기줄의 길이를 정해 두고 넘으면 곧바로 거절하는 방식이 기본이다. 거절은 나쁜 일처럼 보이지만 기다리게 하다가 실패하는 것보다 낫다. 부르는 쪽이 곧바로 다른 선택을 할 수 있기 때문이다.

줄일 때는 무엇을 남길지도 정한다. 모든 요청을 똑같이 거절하는 대신 중요한 요청을 남기고 덜 중요한 요청을 먼저 거절하면, 같은 양을 처리하면서도 더 쓸모 있는 결과가 된다.

부하에 따른 응답의 단계

한 번에 무너지는 대신 단을 밟는다



점선 단계에 닿지 않게 앞의 단을 만든다

4.5 줄여서 주는 응답

저하 상태에서 돌려주는 응답은 정상 응답과 모양이 같아야 다루기 쉽다. 항목의 수가 줄거나 일부 자리가 비어 있되, 구조 자체가 달라지면 받는 쪽이 두 가지 형태를 모두 다뤄야 한다.

비어 있는 자리에는 왜 비었는지를 함께 담는다. 값이 없는 것과 지금 가져오지 못한 것을 구분할 수 있어야 받는 쪽이 다시 시도할지, 그냥 넘어갈지 정할 수 있다. 앞 장에서 다룬 실패 은폐를 피하는 자리가 여기다.

화면에서도 같은 원칙이 선다. 빠진 부분을 아무 표시 없이 없애면 사용자는 원래 없는 것으로 여긴다. 지금 보여 주지 못한다는 짧은 표시 하나가 문의와 오해를 크게 줄인다. 줄인 응답도 계약의 일부이므로 그 모양을 미리 적어 둔다.

4.6 저하 상태를 알리는 방법

저하는 조용히 일어나면 위험하다. 사용자도 모르고 운영하는 쪽도 모르면, 저하된 상태가 정상처럼 굳어 며칠이 지나기도 한다.

그래서 저하가 시작되고 끝나는 것을 기록으로 남기고 지표로 센다. 어느 부분이 언제부터 꺼졌는지 알 수 있어야 하고, 꺼진 시간이 길어지면 사람에게 알려야 한다. 저하는 임시 조치이지 상태가 아니다.

사용자에게 알릴 때는 짧고 구체적으로 적는다. 무엇이 지금 안 되는지, 나머지는 쓸 수 있는지, 다시 보려면 어떻게 하는지를 담는다. 문제가 있다는 말만 있고 무엇이 문제인지 없으면 오히려 전체가 고장난 것처럼 읽힌다. 알리지 않은 저하는 다음에 같은 일이 나도 아무도 알아채지 못한다.

4.7 저하가 굳어질 때

저하는 임시로 버티기 위한 것인데 실제로는 오래 남는 일이 잦다. 꺼 둔 부분이 없어도 큰 불편이 없으면 아무도 다시 켜지 않는다. 몇 달 뒤에는 그 부분이 원래 없던 것처럼 여겨진다.

이 상태가 나쁜 이유는 두 가지다. 꺼진 부분의 코드가 쓰이지 않은 채 남아 남아 가고, 다시 켜야 할 때 켜지지 않는다. 그리고 진짜 문제였던 원인이 그대로 남는다.

그래서 저하에는 기한을 붙인다. 언제까지 이 상태를 두고, 그 뒤에는 원인을 고치거나 그 기능을 정식으로 없애기로 정한다. 기한 없는 임시 조치는 결정을 미루는 것이지 해결이 아니다. 기한을 정해 두면 임시 조치가 결정으로 바뀌는 시점이 생긴다. 임시라는 말은 기한이 적혀 있을 때만 임시다.

5.1 값을 정하는 근거

경계 설계에는 수가 필요하다. 얼마를 기다릴지, 몇 번 다시 할지, 대기줄을 얼마나 둘지가 모두 수다. 이 수를 어림으로 정하면 어림으로 동작한다.

근거로 쓸 만한 것은 셋이다. 정상일 때의 측정값, 감당할 수 있는 자원의 크기, 그리고 사람이 기다릴 수 있는 시간이다. 대기 상한은 첫째와 셋째에서 오고, 대기줄 길이는 둘째에서 온다.

근거가 없는 수는 대개 다른 곳에서 복사해 온 값이다. 다른 곳의 사정에 맞춘 값이 이곳에 맞을 이유가 없다. 값마다 근거를 한 줄로 적어 두면, 나중에 상황이 달라졌을 때 그 근거가 아직 유효한지만 확인하면 된다. 근거가 있는 값은 다른 사람이 이어받아 조정할 수 있다. 근거 한 줄이 값 하나보다 오래 쓸모를 남긴다.

5.2 처음 값과 조정

처음부터 맞는 값을 찾으려 하면 시작할 수 없다. 그래서 근거로 정한 값을 처음 값으로 두고, 실제 동작을 보며 조정한다. 중요한 것은 조정할 수 있는 상태로 시작하는 것이다.

조정하려면 두 가지가 필요하다. 그 값이 걸리는 횟수를 셀 수 있어야 하고, 값을 바꾸기 쉬워야 한다. 상한에 걸린 횟수가 늘 영이면 상한이 너무 큰 것이고, 자주 걸리면 너무 작거나 상대가 느려진 것이다.

조정은 한 번에 하나씩 한다. 대기 상한과 재시도 횟수를 함께 바꾸면 어느 쪽 때문에 달라졌는지 알 수 없다. 값 하나를 바꾸고 지켜보고 다음으로 넘어가는 순서가 느려 보여도 결국 빠르다. 조정할 수 없는 값은 처음 정한 사람이 떠나면 아무도 손대지 못한다.

5.3 값을 실제로 확인하기

값을 정해 두었다고 그 값이 실제로 걸리는 것은 아니다. 상한을 오 초로 두었는데 아래 층에서 삼십 초를 쓰고 있으면 오 초는 걸리지 않는다. 설정한 값과 걸리는 값이 다른 일은 흔하다.

확인하는 방법은 일부러 그 상황을 만들어 보는 것이다. 응답하지 않는 상대를 흉내 내고 실제로 몇 초 만에 끊기는지 쟀다. 재시도 횟수도 같은 방식으로 확인한다. 몇 번 부르는지 세면 정한 값과 맞는지 알 수 있다.

이 확인을 한 번 해 두면 그 뒤로는 값을 바꿀 때마다 다시 하기 쉽다. 확인하지 않은 경계는 있다고 믿는 경계이지 실제 경계가 아니다. 필요할 때 걸리지 않는 상한은 없는 것과 같다. 확인은 한 번 만들어 두면 값을 바꿀 때마다 다시 쓸 수 있다.

5.4 값이 서로 얹힐 때

경계 값들은 따로 놓고 보면 합리적인데 함께 놓으면 어긋나는 일이 있다. 대기 상한이 오 초이고 재시도가 세 번이면 최악의 경우 십오 초가 걸린다. 바깥 요청의 상한이 십 초라면 이 조합은 절대 끝까지 가지 못한다.

그래서 값을 정할 때는 조합의 결과를 계산해 본다. 상한 곱하기 시도 횟수에 쉬는 시간을 더한 값이 바깥 상한 안에 들어오는지 보는 식이다. 이 계산은 종이 위에서 할 수 있고 대부분의 어긋남이 여기서 드러난다.

값 사이의 관계를 적어 두면 나중에 하나를 바꿀 때 다른 값도 함께 보게 된다. 적어 두지 않으면 재시도만 늘리는 변경이 조용히 상한을 넘기게 만든다. 얹힌 값은 함께 적어 두고 함께 바꾼다.

5.5 환경마다 다른 값

같은 값이 모든 환경에서 맞지는 않다. 시험 환경은 자원이 적고 상대도 느리며, 실제 환경은 자료가 많고 동시 요청도 많다. 그래서 경계 값은 환경별로 다르게 두는 것이 보통이다.

다르게 두되 어느 값이 어느 환경에서 무엇인지 한자리에서 볼 수 있어야 한다. 환경마다 흩어져 있으면 실제 환경에만 있는 값을 시험 환경에서 빠뜨리고, 그 차이가 문제로 드러날 때까지 아무도 모른다.

기본값은 가장 조심스러운 환경에 맞춘다. 실제 환경에 맞춰 넉넉하게 잡아 두면, 그 값을 정하지 않은 새 환경에서 자원을 넘겨 쓰게 된다. 기본값은 잘 모르는 자리에서 쓰인다는 원칙이 여기서도 선다. 환경별 값의 차이 자체가 그 환경의 성질을 알려 주는 기록이 된다.

5.6 값이 낡는 방식

경계 값은 정한 시점의 상황에 맞춰진다. 상대가 빨라지거나 느려지고, 자료가 늘고, 자원이 바뀌면 그 값은 조용히 맞지 않게 된다. 값이 낡아도 오류가 나지 않으므로 아무도 알아채지 못한다.

낡음이 드러나는 자리는 두 곳이다. 상한에 걸리는 횟수가 부쩍 늘거나, 반대로 몇 년째 한 번도 걸리지 않는 경우다. 앞쪽은 상황이 나빠진 것이고 뒤쪽은 상한이 사실상 없는 것이다.

그래서 경계 값은 이따금 다시 본다. 볼 때는 정할 때 적어 둔 근거를 먼저 읽고 그 근거가 아직 유효한지 확인한다. 근거가 적혀 있지 않으면 이 확인 자체가 불가능해 값을 그대로 두게 된다. 걸린 횟수를 세어 두면 값이 낡았다는 사실이 수로 드러난다.

5.7 값을 기록하는 자리

경계 값은 코드와 설정과 문서에 흩어지기 쉽다. 흩어지면 어느 것이 실제 값인지 알 수 없고, 근거는 대개 문서에만 남아 값이 바뀔 때 갱신되지 않는다.

실용적인 배치는 값과 근거를 같은 자리에 두고 문서에는 그 자리를 가리키는 것이다. 값이 정의된 곳에 한 줄로 근거를 적으면 값을 고치는 사람이 그 줄을 함께 본다.

여기에 더해 지금 실제로 쓰이는 값을 시작할 때 남긴다. 정의된 값과 쓰이는 값이 다를 수 있으므로, 두 가지가 모두 있어야 확인이 끝난다. 값과 근거와 실제 적용값, 이 셋이 갖춰지면 경계를 다루는 데 필요한 정보가 모두 있는 셈이다. 흩어진 기록은 시간이 지나면 서로 다른 값을 가리키게 된다.

6.1 연동의 상황

이 장은 바깥 시스템 하나와의 연동에 경계를 정한 기록이다. 이 연동은 주문 화면에서 배송 예정일을 받아 오는 데 쓰였고, 평소에는 백 밀리초 안에 답이 왔다.

문제는 상대가 느려진 어느 날 드러났다. 응답이 오지 않는 동안 주문 화면 전체가 멈췄고, 기다리는 요청이 쌓이면서 배송과 무관한 화면까지 열리지 않았다. 상대는 삼십 분 뒤에 돌아왔지만 이쪽은 그 사이 아무것도 하지 못했다.

확인해 보니 대기 상한이 설정되어 있지 않았다. 정하지 않았으므로 연결 도구의 기본값이 쓰이고 있었고, 그 값은 삼십 초에 가까웠다. 아무도 그 값을 고른 적이 없었지만 그 값이 이 시스템의 동작을 정하고 있었다. 정하지 않은 값이 시스템의 동작을 정하고 있었던 셈이다.

6.2 무엇부터 정했나

가장 먼저 정한 것은 대기 상한이다. 근거는 정상일 때의 응답 분포였다. 대부분이 백 밀리초 안에 끝났고 상위 구간이 사백 밀리초였으므로, 그보다 조금 넉넉한 일 초로 정했다.

다음으로 바깥 요청의 상한을 확인했다. 주문 화면 전체의 상한이 삼 초였으므로 안쪽 상한 일 초는 그 안에 들어왔다. 재시도를 한 번 넣어도 이 초 남짓이라 여유가 있었다.

재시도는 한 번만 두기로 했다. 상대가 느려진 상황에서 여러 번 부르면 상대를 더 힘들게 하고, 이 값은 없어도 되는 정보라 굳이 여러 번 시도할 이유가 없었다. 실패 종류를 보고 이쪽 잘못된 경우는 다시 하지 않게 했다. 값을 정하는 데 걸린 시간보다 근거를 확인하는 데 걸린 시간이 길었다.

6.3 없어도 되는 부분 정하기

배송 예정일이 없어도 주문은 할 수 있는지 물었다. 답은 그렇다였다. 예정일은 참고 정보였고, 없으면 주문 뒤에 안내로 알리는 흐름이 이미 있었다.

그래서 이 부분을 저하 대상으로 정했다. 값을 받지 못하면 그 자리만 비우고 나머지 화면은 그대로 그린다. 비운 자리에는 지금 예정일을 불러오지 못했다는 짧은 표시를 넣기로 했다.

여기서 한 가지를 더 확인했다. 예정일 값이 화면 표시 말고 다른 처리에 쓰이는지 보았다. 한 자리에서 이 값으로 배송비를 계산하고 있었다. 그 자리는 저하 대상이 될 수 없으므로 값이 없으면 실패로 다루도록 나눴다. 같은 값이라도 쓰이는 자리에 따라 저하 대상이 되기도 하고 아니기도 한다.

6.4 실패를 어떻게 알렸나

값을 받지 못했을 때 빈 문자열이나 오늘 날짜를 대신 넣는 방법도 있었지만 쓰지 않았다. 앞 장에서 본 실패 은폐가 그대로 일어나기 때문이다. 특히 오늘 날짜는 실제 값처럼 보여 가장 위험했다.

대신 응답의 그 자리에 값이 없음과 그 이유를 함께 담았다. 화면은 그 이유를 보고 예정일 자리에 짧은 안내를 그린다. 받는 쪽 코드는 값이 없는 경우와 지금 가져오지 못한 경우를 구분해 다룰 수 있게 되었다.

운영 쪽으로는 실패 건수를 지표로 썼다. 평소에는 영에 가깝고, 상대가 느려지면 곧바로 는다. 이 지표가 저하 상태에 들어갔다는 사실을 알리는 신호가 되었다. 받는 쪽이 구분할 수 있게 만든 것이 이 작업에서 가장 오래 남았다.

6.5 값이 실제로 걸리는지 확인

정한 값이 실제로 걸리는지 확인했다. 응답하지 않는 상대를 흉내 내고 화면을 열어 몇 초 만에 답이 돌아오는지 잰다.

처음 확인에서는 일 초가 아니라 육 초가 걸렸다. 확인해 보니 연결을 얻는 단계에 별도의 상한이 있었고 그 값이 오 초였다. 상한이 하나가 아니라 둘이었던 것이다. 연결 상한을 이백 밀리초로 낮추자 전체가 일 초 안에 끝났다.

재시도도 세었다. 실패한 요청이 실제로 두 번 나가는지 기록으로 확인했다. 값을 정한 것과 그 값이 걸리는 것은 다르다는 사실을 이 확인에서 두 번 겪었다. 확인하지 않았다면 상한을 정해 두고도 육 초를 기다렸을 것이다. 상한이 둘이라는 사실은 문서 어디에도 적혀 있지 않았다.

6.6 다시 겪은 날

몇 달 뒤 상대가 다시 느려졌다. 이번에는 주문 화면이 정상으로 열렸고 예정일 자리에만 안내가 떴다. 배송비를 계산하는 자리는 실패로 처리되어 그 항목만 주문이 막혔다.

실패 지표가 올라가 알림이 갔고, 상대의 상태를 확인한 뒤 기다렸다. 상대가 돌아오자 지표는 다시 내려갔고 사람이 손댈 일은 없었다.

이때 확인한 것이 하나 있다. 저하 상태가 두 시간 이어졌는데 그동안 화면의 안내 문구가 그대로였다. 오래 이어지면 문구를 바꾸거나 다른 방법을 안내하는 편이 나았을 것이다. 이 점을 다음 개선 후보로 적었다.경계가 있는 상태와 없는 상태의 차이가 같은 상황에서 그대로 드러났다.사람이 손대지 않아도 지나간 장애는 기록으로만 남았다.

6.7 남긴 값과 근거

작업이 끝나고 남긴 것은 값 넷과 그 근거다. 대기 상한 일 초, 연결 상한 이백 밀리초, 재시도 한 번, 저하 대상 하나다. 각각 왜 그 값인지 한 줄씩 적었다.

값은 밖에서 바꿀 수 있게 두었다. 상대의 사정이 달라지면 급히 조절해야 할 값이기 때문이다. 대신 기본값은 지금 정한 값으로 두어 아무것도 설정하지 않은 환경에서도 같은 동작이 되게 했다.

마지막으로 시작할 때 이 값들이 무엇으로 쓰이는지 남기게 했다. 다음에 이 자리를 보는 사람은 값을 찾아 여러 층을 뚫이지 않아도 된다. 이 기록이 이번 작업에서 가장 오래 쓰일 결과물이다. 값 넷과 근거 녀 줄이 다음 사람이 이어받을 수 있는 형태로 남았다.

7.1 얼마나 방어할 것인가

일어날 수 있는 모든 실패에 대비하면 코드가 실패 처리로 가득 찬다. 읽기 어려워지고, 그 처리 자체에 잘못이 생기며, 정작 자주 일어나는 실패를 덮어쓴다.

그래서 대비할 실패를 고른다. 기준은 두 가지다. 실제로 일어나는가, 일어나면 얼마나 넓게 번지는가. 자주 일어나고 넓게 번지는 것부터 경계를 세우고, 드물고 좁게 끝나는 것은 그대로 실패하게 둔다.

대비하지 않기로 한 것도 적어 둔다. 적어 두지 않으면 다음 사람이 빠뜨린 것으로 여기고 다시 검토한다. 대비하지 않는 것은 결정이지 누락이 아니며, 그렇게 적혀 있어야 결정으로 읽힌다. 대비하지 않기로 한 목록이 대비한 목록만큼 쓸모 있을 때가 있다. 결정으로 적힌 미대비는 다음 검토에서 근거만 확인하면 된다.

7.2 조용한 실패를 없애기

이 책 전체에서 가장 자주 나온 실수는 실패를 조용히 넘기는 것이다. 빈 값으로 대신하고, 예외를 삼키고, 저하를 알리지 않는 일이 모두 여기 든다. 대신하는 값과 실제 값을 구분할 수 없게 만드는 것이 가장 나쁜 선택이다.

실패를 보이게 만드는 방법은 셋이다. 실패를 값으로 돌려주되 실패임을 알 수 있게 하고, 실패의 수를 세고, 실패가 이어지면 사람에게 알린다. 셋 다 실패를 없애지는 않지만 실패가 숨지 못하게 한다.

보이게 만들면 고칠 기회가 생긴다. 숨은 실패는 몇 달 뒤 결과가 이상해진 뒤에야 드러나고, 그때는 원인을 찾는 데 훨씬 큰 비용이 든다. 실패를 드러내는 일은 실패를 늘리는 것이 아니라 이미 있던 실패를 보이게 하는 것이다.

7.3 기본값을 바꾸는 순서

기본값을 바꾸면 그 값을 정하지 않은 모든 자리의 동작이 함께 바뀐다. 설정을 바꾸는 것보다 영향이 넓은데도 코드 안의 값이라 가볍게 바뀌는 일이 잦다.

그래서 순서를 둔다. 먼저 지금 이 값을 명시적으로 설정한 자리가 얼마나 되는지 센다. 대부분이 설정하고 있다면 기본값 변경의 영향은 작다. 대부분이 기본값을 쓰고 있다면 영향이 크므로 알리고 바꾼다.

바꾼 뒤에는 새 값이 걸리는 자리를 지켜본다. 상한을 줄였다면 걸리는 횟수가 늘 것이고, 그 증가가 예상 범위 안인지 확인한다. 기본값 변경은 되돌리기 쉬운 상태로 해 두어야 이 확인이 의미를 갖는다. 기본값은 조용히 넓게 퍼지므로 알리는 절차가 특히 중요하다.

7.4 경계 설계 점검 항목

경계 설계를 점검할 때는 다음을 본다. 첫째, 바깥을 부르는 모든 자리에 기다림의 상한이 있는가. 없으면 그 자리가 전체를 멈출 수 있는 자리다.

둘째, 안쪽 상한이 바깥 상한보다 짧은가. 셋째, 재시도가 어떤 실패에만 일어나는가. 넷째, 실패했을 때 돌려주는 값이 실제 값과 구분되는가.

다섯째, 이 값들이 실제로 걸리는지 확인해 본 적이 있는가. 마지막 항목이 가장 자주 빠진다. 앞의 넷은 적혀 있는 것을 보는 일이지만 다섯째는 만들어 보아야 알 수 있다. 걸리는 항목이 나오면 그 항목을 다룬 절로 돌아간다. 다섯 항목은 새 연동을 붙일 때 그대로 다시 쓰인다. 빠뜨리기 쉬운 항목일수록 목록에 남겨 두는 값이 크다.

7.5 값을 다시 볼 때

경계 값은 한 번 정하면 오래 남는다. 그래서 다시 보는 시점을 정해 둔다. 상대가 바뀌었을 때, 자료나 요청이 크게 늘었을 때, 그리고 아무 일이 없어도 이따금이다.

볼 때는 먼저 그 값이 걸린 횟수를 본다. 한 번도 걸리지 않았다면 상한이 사실상 없는 것이고, 자주 걸린다면 값이 작거나 상황이 나빠진 것이다. 이 수가 없으면 재검토는 짐작이 된다.

다음으로 정할 때 적어 둔 근거를 읽는다. 근거가 여전히 맞으면 값도 그대로 두고, 근거가 달라졌으면 새 근거로 값을 다시 계산한다. 값을 바꾸는 것보다 근거를 갱신하는 일이 중요하다. 근거가 맞으면 값은 따라온다. 재검토 시점을 정해 두지 않으면 값은 사실상 영구히 남는다.

7.6 저하와 중단 사이

모든 상황에서 줄여서 버티는 것이 옳지는 않다. 잘못된 결과를 주느니 멈추는 편이 나은 자리가 있다. 돈이나 재고처럼 어긋나면 되돌리기 어려운 값이 그렇다.

그래서 자리마다 정한다. 부분이 없어도 사용자가 하려던 일을 끝낼 수 있으면 줄여서 주고, 그 부분이 결과의 정확성을 정한다면 멈춘다. 6장 사례에서 예정일은 앞쪽이었고 배송비 계산은 뒤쪽이었다.

판단이 서지 않으면 멈추는 쪽을 고른다. 멈추면 사용자가 알고 다시 시도하지만, 잘못된 결과는 알지 못한 채 남는다. 저하는 정확성을 해치지 않는 자리에서만 쓰는 수단이다. 어느 쪽을 골랐는지 적어 두면 다음 사람이 같은 고민을 다시 하지 않는다. 이 판단은 기술 결정이면서 동시에 업무 결정이다.

7.7 핵심 원칙 정리

이 책이 반복한 물음은 둘이다. 아무도 정하지 않으면 무엇이 쓰이는가, 그리고 잘 안 될 때 어디까지 번지는가. 앞의 물음이 기본값을 정하고 뒤의 물음이 경계를 정한다.

나머지는 두 물음의 도구다. 설정 총은 값이 어디서 오는지 말할 수 있게 하고, 상한과 재시도는 기다림이 쌓이지 않게 하며, 점진적 저하는 한 부분의 실패가 전체가 되지 않게 한다.

끝으로 덧붙일 것은 정하지 않은 것도 결정이라는 점이다. 상한을 정하지 않으면 도구의 기본값이 이 시스템의 동작을 정하고, 저하 순서를 정하지 않으면 문제가 생긴 순간의 판단이 그것을 정한다. 미리 정해 두는 일이 이 책이 다룬 설계의 전부다. 정하지 않은 자리를 찾아 하나씩 정해 가는 것이 이 작업의 실제 모습이다.